

Q4.1 Define the three types of I/O. Identify each as either microprocessor-initiated or device-initiated.

There are three types of I/O used in microcomputer systems:

1. Programmed I/O (Microprocessor-Initiated)

In programmed I/O, the microprocessor controls all data transfer using a program stored in memory. The microprocessor decides when to send or receive data. Because the CPU starts and controls everything, this is microprocessor-initiated I/O.

There are two types of programmed I/O:

- Unconditional Programmed I/O: The microprocessor sends data without checking if the device is ready. The device must always be ready to accept data.
- Conditional Programmed I/O: The microprocessor checks if the device is ready before transferring data. It waits in a loop until the device signals it is ready.

2. Interrupt I/O (Device-Initiated)

In interrupt I/O, the external device tells the microprocessor that it is ready by sending an interrupt signal through the INT pin. The microprocessor stops its current work, executes an Interrupt Service Routine (ISR), and then returns to the main program. Since the device starts the process, this is device-initiated I/O.

3. Direct Memory Access – DMA (Device-Initiated)

In DMA, data is transferred directly between memory and an I/O device without the microprocessor. A special DMA controller chip handles the transfer. The I/O device requests the DMA transfer, so this is also device-initiated I/O.

I/O Type	Who Starts It?	Initiated By
Programmed I/O	Microprocessor runs a program	Microprocessor
Interrupt I/O	Device sends interrupt signal	Device
DMA	Device requests DMA via controller	Device

Q4.2 What is the basic difference between standard I/O and memory-mapped I/O? Identify the programmed I/O technique(s) used by Intel and Motorola microprocessors.

Feature	Standard I/O	Memory-Mapped I/O
Also called	Port I/O or Isolated I/O	Memory-Mapped I/O
Special control pin	Yes – uses M/IO pin (HIGH = memory, LOW = I/O)	No – no special pin needed

Feature	Standard I/O	Memory-Mapped I/O
How I/O is addressed	Uses a separate I/O address space	I/O ports are mapped as memory addresses
Instructions used	IN and OUT instructions	Memory instructions like MOV
Address size	Usually 8-bit (up to 256 ports)	Uses part of full memory address space
Used by	Intel microprocessors	Motorola 68000 / 68020; also Intel

Intel Microprocessors:

Intel processors (such as the Pentium) can use both standard I/O and memory-mapped I/O. Standard I/O uses the IN and OUT instructions. Memory-mapped I/O uses MOV instructions.

Motorola Microprocessors:

Motorola processors (such as the 68000 and 68020) do not have a special I/O control pin like M/IO. So they use only memory-mapped I/O.

Q4.3 What are programmed I/O and virtual I/O?

Programmed I/O:

In programmed I/O, the microprocessor controls all data transfer between itself and external devices using a program. The microprocessor communicates through special registers called I/O ports. Everything is controlled by the software running in the CPU.

There are two forms:

- Unconditional I/O: Data is sent without checking if the device is ready. Used for devices that are always ready (e.g., seven-segment display).
- Conditional I/O: The microprocessor checks the device status before sending or receiving data. It uses handshaking signals.

Virtual I/O:

Virtual I/O is used when an operating system (OS) is present. The user does not deal directly with the physical hardware. Instead, the OS acts as a middle layer between the user and the actual I/O devices.

The OS creates virtual (logical) devices. For example, a user program may write output to a virtual printer (which is really disk space). The OS later sends this output to an actual physical printer. This is called spooling.

Virtual I/O offers these advantages:

- The user does not need to know hardware details.
- The program does not need to be changed if a different physical device is used.
- System speed and flexibility improve.

Q4.4 *What is the difference between memory mapping in a microprocessor and memory-mapped I/O?*

Feature	Memory Mapping in a Microprocessor	Memory-Mapped I/O
What it is	Assigning address ranges to different memory chips (ROM/RAM) connected to the CPU	Assigning some memory addresses to I/O devices (ports) instead of actual memory
Purpose	To organize physical memory (RAM, ROM) in the address space	To allow I/O ports to be accessed using memory instructions
What occupies the address	Actual physical memory chips (RAM or ROM)	I/O port registers (which may not be real memory)
Instructions used	MOV and other memory instructions to read/write data	MOV and other memory instructions used for I/O operations
Example	ROM placed at address 0000 to 3FFF, RAM at 4000 to 7FFF	I/O port at address 8000 treated as a memory location

In short: Memory mapping organizes real memory chips in the address space. Memory-mapped I/O uses some of those address slots for I/O devices rather than memory.

Q4.5 *Discuss the basic difference between polled I/O and interrupt I/O.*

Feature	Polled I/O (Conditional Programmed I/O)	Interrupt I/O
Who starts it?	Microprocessor	External device
How it works	CPU keeps checking the device status in a loop until it is ready	Device sends interrupt signal (INT) when it is ready
CPU time	Wasted – CPU is stuck in a loop waiting	Saved – CPU continues other work until interrupted
Speed	Slow – especially for slow devices	Faster – no waiting loop
Suitability	Good for fast devices	Good for slow devices
Example	CPU checking A/D converter conversion complete signal	A/D converter interrupts CPU when conversion is done

The key difference: In polled I/O, the CPU wastes time waiting and checking. In interrupt I/O, the CPU is free to do other work and only stops when the device tells it to.

Q4.6 *What is the difference between subroutine and interrupt I/O?*

Feature	Subroutine Call	Interrupt I/O
Who starts it?	The main program (microprocessor)	External device (device-initiated)
When does it happen?	At a fixed point decided by the programmer	Any time – device can interrupt at any moment
Called how?	Using a CALL instruction in the program	Device activates the INT pin on the CPU
Predictable?	Yes – the programmer knows when it runs	No – timing is unpredictable
Return instruction	RET (Return from subroutine)	RTE (Return from Exception / Interrupt)
Registers saved	Only registers the programmer saves manually	PC and Status Register (SR) are saved automatically by hardware
Use case	To run a known piece of code at a known time	To handle events from external devices like keyboards or A/D converters

In short: A subroutine is planned and called by the programmer. An interrupt is unplanned and triggered by an external device at any time.

Q4.7 What is an interrupt address vector?

An interrupt address vector is the starting memory address of the Interrupt Service Routine (ISR). When an interrupt occurs, the microprocessor needs to know where to go in memory to find the program that handles the interrupt. This address is called the interrupt address vector.

There are two methods to provide the interrupt address vector:

1. Fixed Address Method:

The manufacturer of the microprocessor defines a fixed memory address for each interrupt. The ISR must be placed at this fixed address by the programmer.

2. Indirect Method:

The interrupt address vector is stored at a fixed memory location decided by the manufacturer. When an interrupt occurs, the CPU reads that fixed memory location to find the actual ISR address. This allows the programmer to place the ISR anywhere in memory.

Example (from the textbook): In a 68000-based system, when an interrupt occurs, the CPU jumps to address \$4000, where the ISR is stored. This address (\$4000) is the interrupt address vector.

Q4.8 Summarize the basic difference between maskable and nonmaskable interrupts.

Describe how power failure interrupt is normally handled.

Feature	Maskable Interrupt	Non-Maskable Interrupt (NMI)
Can it be disabled?	Yes – can be enabled or disabled by software	No – cannot be disabled by any instruction
Priority	Lower priority	Higher priority than maskable interrupts
Control instructions (Intel Pentium)	CLI disables it; STI enables it	No instruction can disable it
Used for	General device interrupts (keyboards, A/D converters, etc.)	Critical events – especially power failure
When both occur together	Serviced after non-maskable interrupt	Serviced first

How Power Failure Interrupt is Handled:

Most microprocessors use +5 V DC power converted from 110 V AC. If the AC voltage drops below 90 V, the +5 V DC cannot be maintained. However, it takes a few milliseconds for the power to actually drop below this level.

During this short time window, the power failure sensing circuit sends a Non-Maskable Interrupt (NMI) to the microprocessor. The CPU responds immediately and runs a special ISR that:

- Saves important (critical) data to non-volatile memory (such as battery-backed CMOS RAM).
- Ensures no data is lost when power comes back.

When power returns, the program can continue from where it stopped without any loss of data.

Q4.9 Why are polled interrupt and daisy chain interrupt used?

A microprocessor usually has only one or a few interrupt pins. But in a real system, many devices may need to interrupt the CPU at the same time. Since all devices share the same interrupt line, the CPU must figure out which device interrupted it. Polled interrupt and daisy chain interrupt are two methods to handle this problem.

Polled Interrupt (Software-Based):

The CPU jumps to a general ISR and checks (polls) each device one by one to find who caused the interrupt. The device checked first gets the highest priority.

- Simple to implement – only needs software.
- Slow – polling all devices takes time.
- Best for systems with few devices.

Daisy Chain Interrupt (Hardware-Based):

Devices are connected in a chain. When an interrupt occurs, the CPU sends an Interrupt Acknowledge (INTA) signal to the first (highest priority) device. If that device did not interrupt, it

passes INTA to the next device, and so on until the correct device is found. That device then sends the interrupt address vector.

- Faster than polling – uses hardware logic.
- Priority is determined by position in the chain (nearest = highest priority).
- More hardware is needed.

Feature	Polled Interrupt	Daisy Chain Interrupt
Method	Software	Hardware
Speed	Slow	Fast
Priority setting	Order of polling in program	Physical position in chain
Hardware needed	Simple	More complex
Best for	Few devices	Many devices

Q4.10 *Discuss the basic difference between internal and external interrupts.*

Feature	External Interrupts	Internal Interrupts (Traps)
Source	Generated by external hardware devices (e.g., keyboard, A/D converter)	Generated inside the microprocessor itself
Cause	Device is ready or needs service	Error conditions inside the CPU (e.g., divide by zero, overflow, illegal opcode)
Who triggers it?	External device activates the INT pin	CPU detects an error and triggers internally
Can it be masked?	Maskable interrupts can be disabled; NMI cannot	Cannot be masked – always handled
Types	Maskable and Non-Maskable (NMI)	Traps (overflow, divide by zero, illegal instruction)
Example	A/D converter or keyboard interrupting the CPU	Division by zero during program execution

In both cases, the CPU stops the current program, saves the PC and SR, and executes a service routine. The programmer must write service routines for both types to handle errors or device requests.

Q4.11 *What are cycle stealing, block transfer, and interleaved DMA?*

These are the three types of DMA (Direct Memory Access) data transfer methods:

1. Block Transfer DMA

The DMA controller takes full and complete control of the system bus. It transfers an entire block of data between memory and the I/O device in one go. During this time, the microprocessor cannot use the bus and can only perform internal operations that do not need the bus.

- Fastest DMA method.
- Most commonly used.
- Best for large data transfers (e.g., hard disk to RAM).
- CPU is completely locked out of the bus until transfer is done.

2. Cycle Stealing DMA

Data is transferred one word (one unit) at a time. The DMA controller steals one clock cycle from the microprocessor, transfers one word, and returns control to the CPU. This is repeated for each word.

It uses an INHIBIT signal:

- INHIBIT HIGH: CPU runs normally.
- INHIBIT LOW: CPU is stopped for one cycle; DMA transfers one word.
- The CPU does not know this happened – the cycle is silently stolen.
- Slows the CPU slightly but does not completely stop it.
- Transfer happens gradually over time.

3. Interleaved DMA

The DMA controller uses the bus only when the CPU is not using it. For example, the CPU does not need the bus when incrementing the Program Counter or when performing ALU (calculation) operations. The DMA controller identifies these idle bus cycles and uses them for data transfer.

- The CPU is never stopped.
- No interference with CPU operations.
- Data transfer is spread over time.
- Logic is more complex to implement.

Feature	Block Transfer	Cycle Stealing	Interleaved
Bus control	DMA takes full control	DMA steals one cycle at a time	DMA uses idle cycles only
CPU stopped?	Yes – for entire block	Yes – one cycle at a time	No
Speed	Fastest	Moderate	Slow but non-disruptive
Complexity	Simple	Moderate	Most complex
Best for	Large bulk data	Continuous small transfers	When CPU must not stop

Q4.12 Summarize the typical functions performed by a DMA controller chip.

A DMA controller chip is a special hardware chip that handles direct data transfer between memory and I/O devices without involving the microprocessor. Its typical functions are:

1. **Accepting DMA Request:** The I/O device requests a DMA transfer through the DMA request line of the controller chip.
2. **Sending HOLD Signal to CPU:** The DMA controller sends a HOLD signal to the microprocessor, asking it to release the system bus.
3. **Receiving HLDA (Hold Acknowledge):** The microprocessor confirms the release of the bus by sending an HLDA (Hold Acknowledge) signal back to the DMA controller.
4. **Taking Control of the Bus:** The DMA controller takes over the address, data, and control buses. It uses its internal address register to place the correct memory address on the address bus.
5. **Sending DMA Acknowledge:** The controller sends a DMA ACK (acknowledge) signal to the I/O device, telling it to start data transfer.
6. **Performing Data Transfer:** The DMA controller manages data transfer directly between RAM and the I/O device, without CPU involvement.
7. **Counting Words Transferred:** The DMA controller uses its terminal count register (counter) to keep track of how many words have been transferred. When the count reaches zero, the transfer is done.
8. **Interrupting the CPU (After Completion):** When the data transfer is complete, the DMA controller sends an interrupt signal (INT) to the CPU to notify it.
9. **Releasing the Bus:** The DMA controller releases the bus by disabling the HOLD and DMA ACK signals. The microprocessor resumes normal operation.

Registers Inside the DMA Controller:

- **Address Register:** Holds the starting memory address for the data transfer.
- **Terminal Count Register:** Holds the number of words (data units) to be transferred.
- **Status Register:** Contains information about the DMA transfer, such as whether it is complete or if an error occurred.

Both the address register and the terminal count register are initialized (set up) by the microprocessor before DMA begins. After that, the DMA controller handles everything automatically.